

Noémie-Alexia Prévost

420-J13-AS

Groupe 1171

RAPPORT D'ÉVALUATION FINALE

Dans le cadre du cours

Structure de données avancées

Travail présenté à :

M. Felix Soumholphakdy

Remis le :

Vendredi 2 mai 2025

Introduction

Whispers In The Dark est un jeu d'horreur solo à la première personne, développée avec Unreal Engine 5. Le jeu se déroule dans un labyrinthe généré procéduralement, dont le chemin change à chaque nouvelle itération du jeu. Le but de cette fonctionnalité est d'ajouter un élément d'imprévisibilité au jeu, puisqu'on ne peut pas se rappeler du chemin entre chaque nouvelle partie. L'objectif du joueur est de naviguer d'un coin du labyrinthe jusqu'à sa sortie, tout en évitant un monstre.

L'ambiance du jeu est sombre et inquiétante. La mécanique principale est liée au son. Le joueur est équipé d'une lampe de poche qu'il peut allumer et éteindre en appuyant sur F. Puisque le jeu est très sombre, cette lampe de poche est essentielle au joueur, mais elle vient avec un prix : lorsqu'elle est allumée, le monstre peut détecter le joueur et le poursuivre. À l'inverse, éteindre la lampe de poche plonge le joueur dans le noir presque total, mais empêche le monstre de suivre la position du joueur. Cette mécanique crée une tension et oblige à utiliser la lumière de manière stratégique.

Avoir la lampe de poche éteinte ne fait pas seulement empêcher le monstre de savoir où le joueur se trouve, il lui permet aussi d'entendre des murmures de certaines portes qui l'aident à naviguer le labyrinthe. Quelques portes qui se trouvent vers le bon chemin pour accéder à la sortie émettent des murmures, mais ceux-ci s'entendent seulement lorsque la lampe de poche est éteinte. Ces chuchotements s'amplifient à mesure que le joueur s'approche. Cette mécanique encourage encore plus de naviguer le jeu sans repère visuel, ce qui ajoute de la tension, en plus des bruits épeurants provenant des portes.

Le labyrinthe lui-même est généré procéduralement grâce à un algorithme personnalisé. Le système commence par une grille ouverte et place des murs ou des portes de manière itérative. L'algorithme s'assure que chaque labyrinthe est unique et cohérent en faisant des vérifications de *path finding*, et ajoute des portes ou des murs tant que celui-ci est toujours fonctionnel.

Le jeu inclut également des fonctionnalités améliorant le *gameplay* tels que des interactions animées avec les portes, du bruitage de pas, des sons d'ouverture et de fermeture de portes, un écran de mort qui apparaît lorsque le monstre attrape le joueur et un tutoriel.

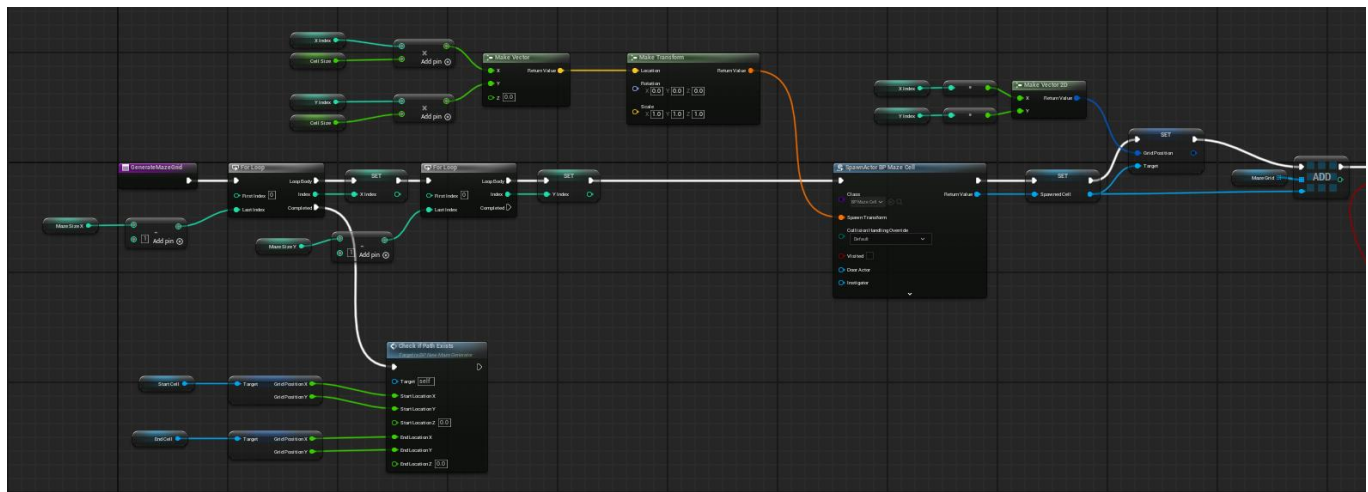
Ce jeu est un mélange de génération procédurale, d'intelligence artificielle interactive et d'exploration avec un élément de risque et récompense. Certaines fonctionnalités ne sont pas encore instaurées, et celles-ci seront détaillées dans la fin du rapport, mais seront fonctionnelles dans la semaine suivant la remise de ce travail.

Tout d'abord, voici les détails de l'intégration d'une liste, d'une table de hachage, d'un graph, ainsi que de trois algorithmes utilisés lors de la création du jeu qui mettent en valeur les connaissances apprises durant le cours de structure de données avancées.

Liste

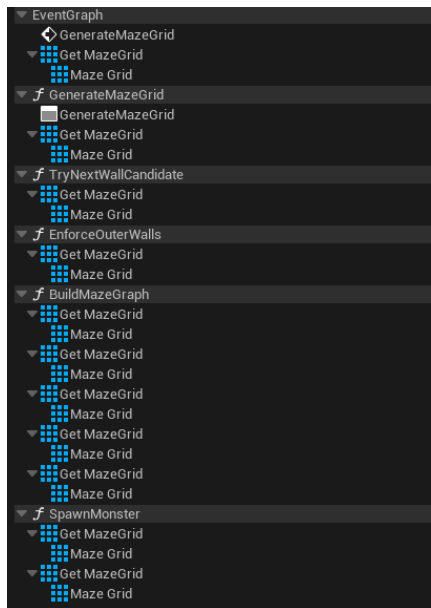
La liste utilisée est représentée par un tableau 2D se nommant MazeGrid. Celle-ci représente la position des cellules du labyrinthe. Elle offre une structure efficace pour la modélisation en grille du labyrinthe. Dans ce jeu, le labyrinthe est composé d'une série de cellules uniformes alignées sur une grille fixe. Chaque cellule est représentée par une tuile de plancher, et 4 murs invisibles y sont rattaché. L'état de cette cellule doit être consulté et modifié fréquemment pendant la génération du labyrinthe et même après pendant la recherche du chemin par l'intelligence artificielle et le placement du monstre. Cette liste est donc indispensable à la logique du jeu.

Elle se crée avec la fonction GenerateMazeGrid() :



Chaque cellule est un acteur (BP_MazeCell) qui remplit la grille. Lors de la fonction GenerateMazeGrid(), le système remplit la liste avec les instances de ces cellules ainsi que leur position. De cette manière, nous pouvons facilement aller chercher dans la liste quelle cellule utiliser pour qu'elle remplisse son rôle (par exemple, le point de départ et la sortie). Chaque membre de la liste possède des références à son index de ligne et colonne. L'accès rapide à des cellules spécifiques, leur modification temporaire et leur restauration seraient bien plus difficiles dans d'autres structures de données.

Voici une capture d'écran de chaque utilisation de cette liste (pour l'instant) :



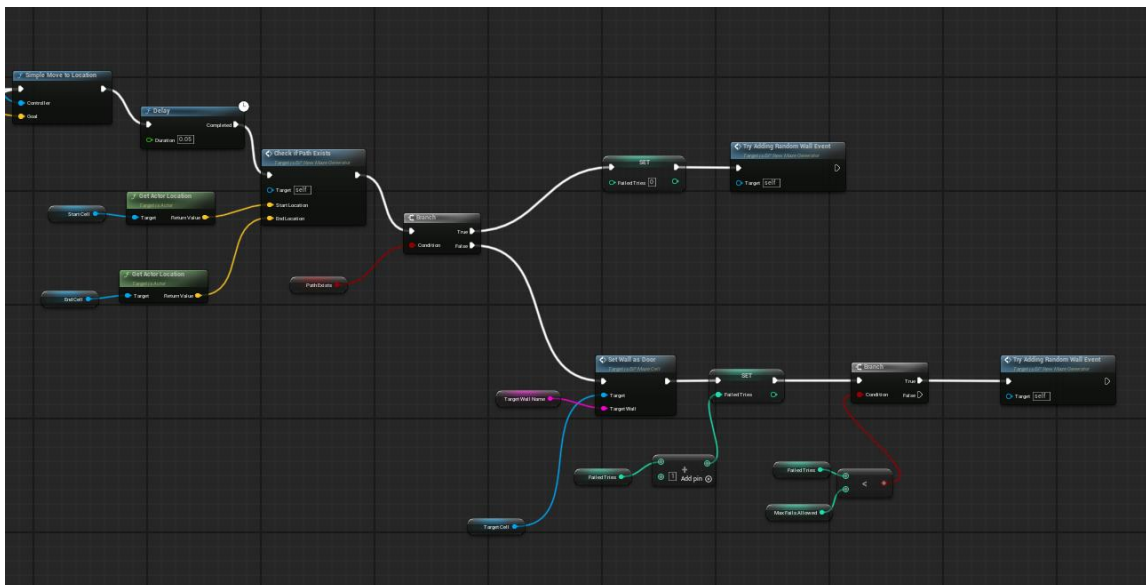
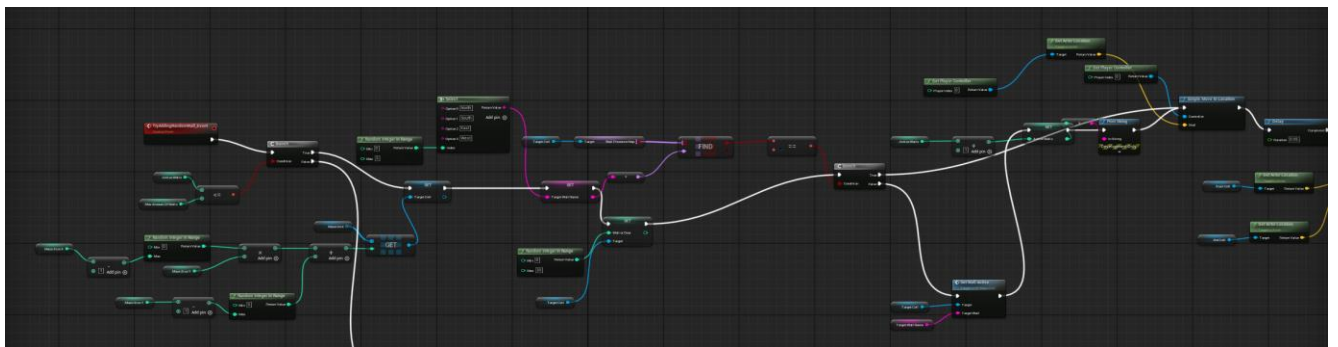
Ce tableau est optimal pour les raisons suivantes :

- Elle donne accès en tout temps à toutes les cellules via ses coordonnées et son nom.
- Elle s'adapte au processus de génération procédurale qui nécessite des vérifications fréquentes des grilles.
- Stocker chaque cellule dans une variable distincte serait extrêmement désordonné et énorme.
- Elle permet une conception modulaire et réutilisable.
- Elle évite les erreurs hors limites grâce à des vérifications d'index valides.
- Elle prend en charge efficacement les fonctions telles que `CheckIfPathExists()`

Initialement, le labyrinthe utilisait un algorithme de retour arrière récursif sur un tableau. Cependant, cette méthode entraînait des problèmes dans le placement des portes. Ce système a été révisé pour commencer avec une grille ouverte et ajouter des murs uniquement lorsque le *pathfinding* le permet. Ce processus repose fortement sur la grille du labyrinthe. Le débogage et le remaniement complet du générateur ont nécessité l'inspection de cellules spécifiques. Ceci a été grandement aidé par la structure fixe et indexée d'un tableau 2D.

Premier algorithme

Pour générer procéduralement un labyrinthe qui se résout et qui est non linéaire, un algorithme de placement de murs aléatoires a été implémenté. Cette conception part d'un espace complètement ouvert et ajoute stratégiquement des murs un par un. L'objectif de cet algorithme est d'introduire un défi en générant un nouveau labyrinthe à chaque instance de jeu. Par contre, celui-ci doit préserver la solvabilité du labyrinthe entre le début et la fin de celui-ci.



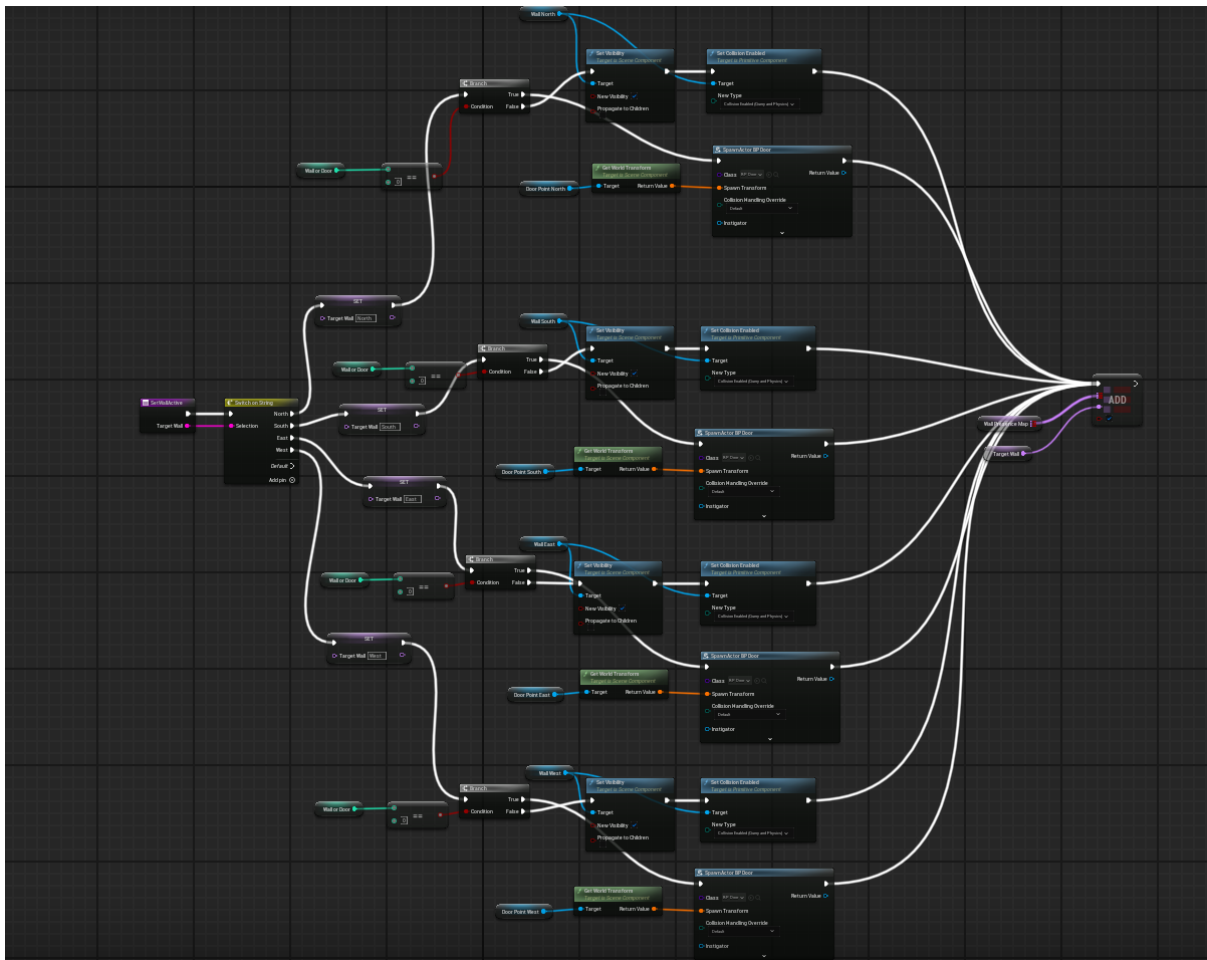
L'algorithme se compose des étapes suivantes :

1. La grille est initialement ouverte sans murs intérieurs.
2. Toutes les positions intérieures de la grille sont parcourues et nous créons une liste de placement de mur possible qui ne sont pas hors du périmètre et qui font parties des cellules du début à la fin.
3. Une cellule aléatoire est choisie et un mur ou une porte est placée temporairement (celui-ci est décidé aléatoirement, il y a 20 chances sur 21 que ce soit un mur qui soit placé au lieu d'une porte, afin qu'il n'y ait pas trop de portes dans le labyrinthe).
4. La solvabilité du chemin est vérifiée grâce au système de navigation d'Unreal.
5. Si un chemin existe toujours, le mur est conservé. Si le mur bloque tous les chemins valides, celui-ci est remplacé par une porte.
6. Cet algorithme recommence, jusqu'à ce qu'on ait atteint 1000 murs ou portes dans la grille.

Cet algorithme a été adopté afin de contrôler le nombre de murs du labyrinthe, en plus de donner une solvabilité garantie et une itération simple et rapide. Le système est également flexible, on peut facilement ajuster le nombre de murs potentiels ou exclure certaines tuiles et il est aussi facilement adaptable afin de modifier la difficulté. Cet algorithme peut sembler simple, mais il est pratique, évolutif et intègre parfaitement les mécanismes du jeu.

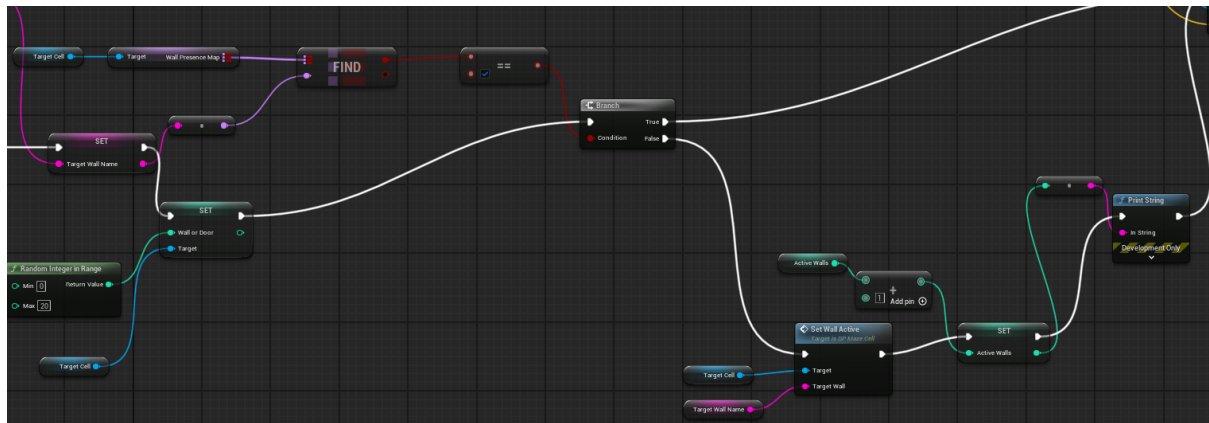
Table de hachage

Lors du développement de Whispers In The Dark, une structure basée sur une table de hachage a été implementée, appelée WallPresenceMap. Celle-ci suit la présence ou l'absence de murs entre les cellules adjacentes. Cette structure joue un rôle crucial dans la manipulation dynamique des murs, le placement des portes et la logique d'interaction. La table de hachage nous permet d'interroger et de mettre à jour efficacement l'état des murs pendant la génération du labyrinthe et le jeu, sans analyser la grille entière ni dupliquer les données.



Chaque clé de WallPresenceMap représente un mur unique entre deux cellules adjacentes. Pour éviter de dupliquer les murs partagés (par exemple : mur nord de la cellule A à mur sud de la cellule B), les clés sont triées à l'aide d'un codage directionnel. La valeur (bool) indique si un mur ou une porte existe déjà entre ces deux cellules. Cette conception permet de stocker les informations sur les murs et portes sans avoir à créer une vingtaine de variables différentes (par exemple : MurNordPlacé, PorteNordPlacée, MurSudPlacé, PorteSudPlacée, ...). Ceci permet également l'ajout et la suppression des murs sans modifier directement le MazeGrid.

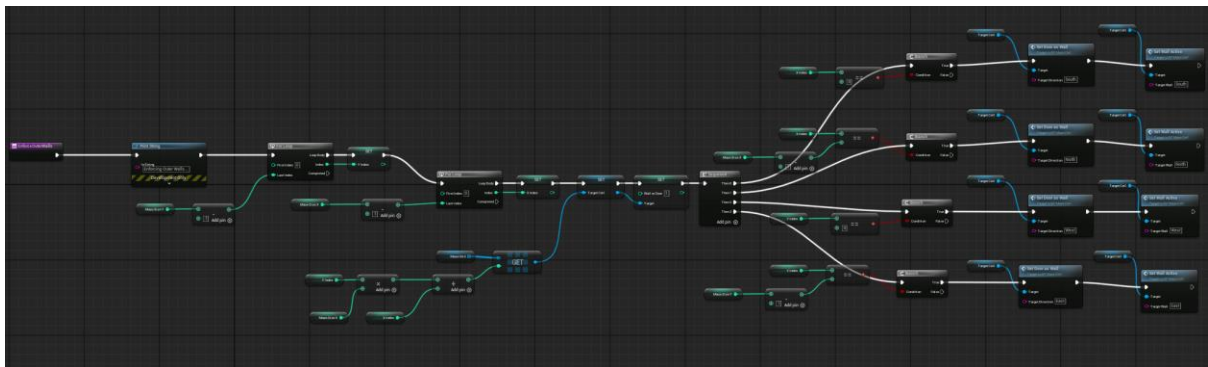
Une table de hachage a aussi été choisie parce qu'elle empêche la désynchronisation entre les cellules, comme par exemple, si une cellule pense qu'un mur existe et l'autre non.



Pour comparer avec d'autres structures afin de justifier la table de hachage, nous pouvons voir qu'une liste 2D de booléens aurait été dans l'impossibilité de représenter un mur entre deux cellules ; les drapeaux de mur par cellule auraient engendré des problèmes de duplication et de synchronisation ; un graph aurait été trop complexe pour une logique de mur uniquement et est plus adapté au *path finding*. La table de hachage était la solution la plus pratique et la plus performante pour représenter les données de murs de façon dynamique.

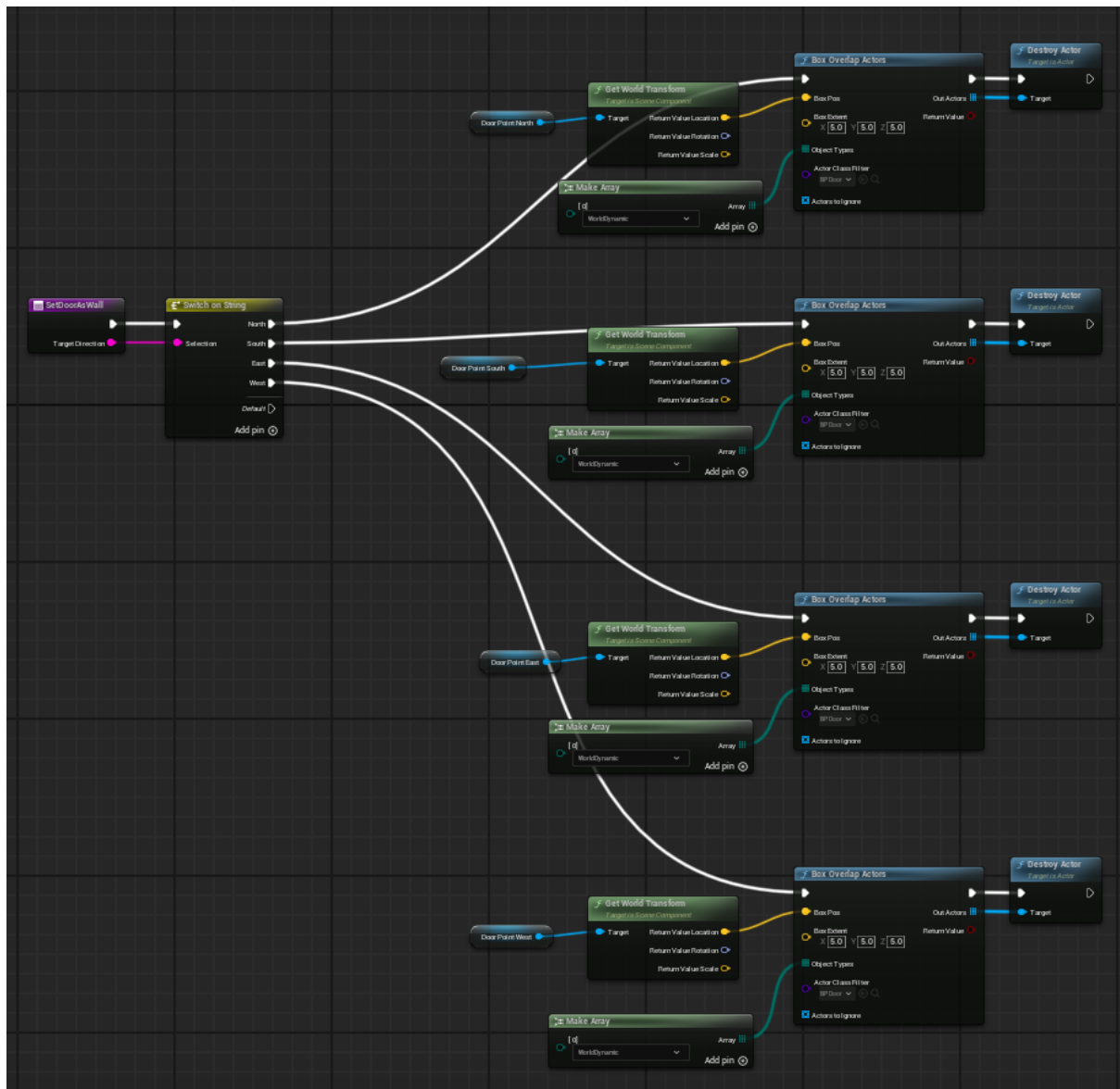
Deuxième algorithme

Afin de garantir que le joueur et le monstre restent dans les limites du labyrinthe généré, un algorithme personnalisé de renforcement des murs extérieurs a été créé. Cet algorithme parcourt dynamiquement les bords extérieurs de la grille et garantit la présence d'un mur solide le long de chaque case limite. Sans cette étape de renforcement, le labyrinthe risquerait d'être exposé au monde extérieur à cause d'une porte placée à cet endroit ou si la génération des 1000 murs n'a pas mis de mur à cette instance. Ceci ferait que le joueur pourrait tomber dans le vide. Cet algorithme fonctionne indépendamment des contraintes du moteur de jeu ou des murs codés, ce qui le rend flexible et évolutif pour s'adapter à toutes les tailles de labyrinthe.



La fonction `EnforceOuterWalls()` suit un modèle de boucles double. Elle parcourt chaque cellule du labyrinthe, et pour chaque cellule, elle évalue si elle se trouve le long de l'une des quatre limites extérieures : Nord, Sud, Est ou Ouest. Si la cellule est située sur l'un de ces bords, un mur est placé du côté correspondant à l'aide de la fonction `SetWallActive()`, qu'on utilise précédemment pour placer des murs aléatoires. Cette fonction active la visibilité et la collision des murs invisibles des cellules.

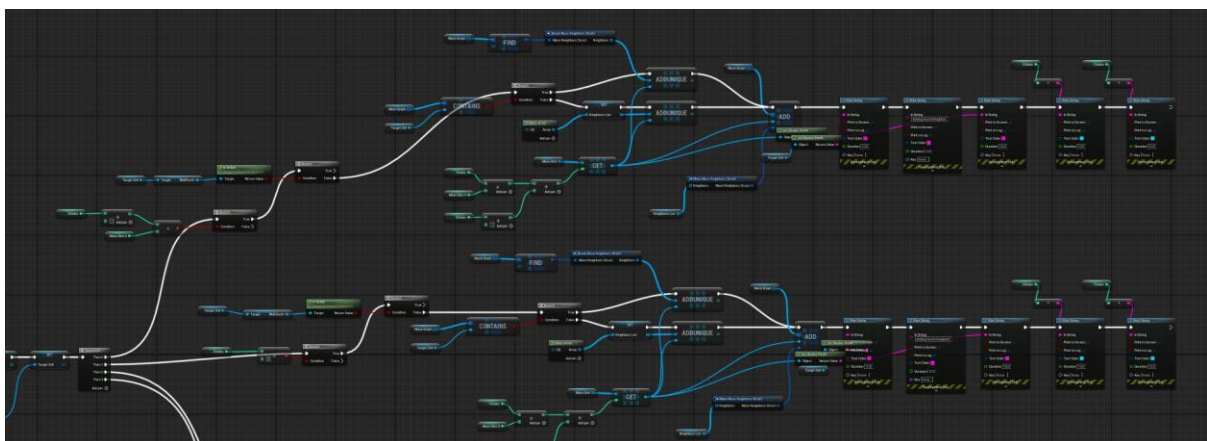
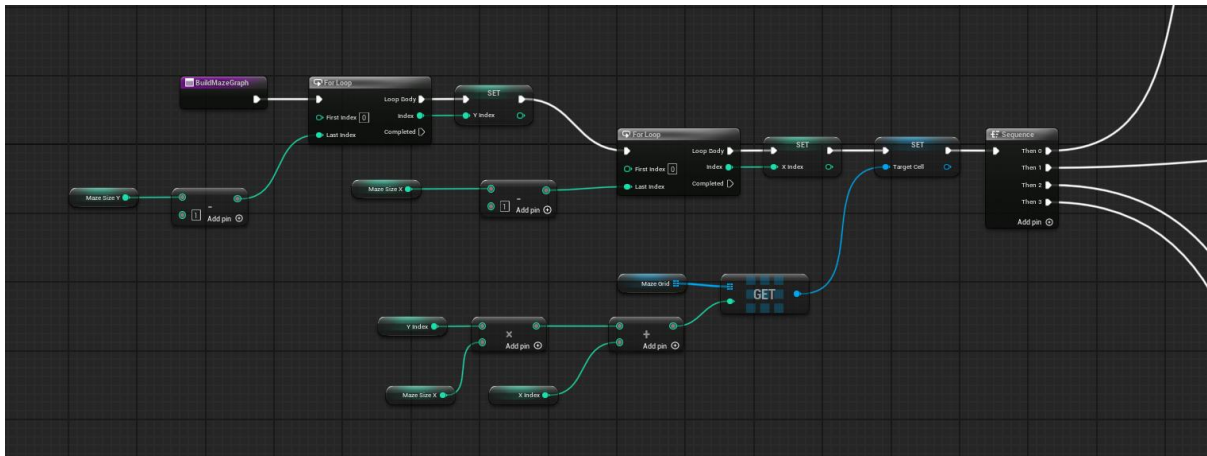
Si une porte existe déjà à cet endroit, tous les acteurs de porte à cet emplacement sont détruits grâce à un *Box Overlap Actor* dans `SetDoorAsWall()`, ce qui empêche les portes de rester le long des murs extérieurs. Ceci est essentiel lors de la génération procédurale où des portes peuvent être ajoutées aux murs intérieurs plus tôt dans le processus.

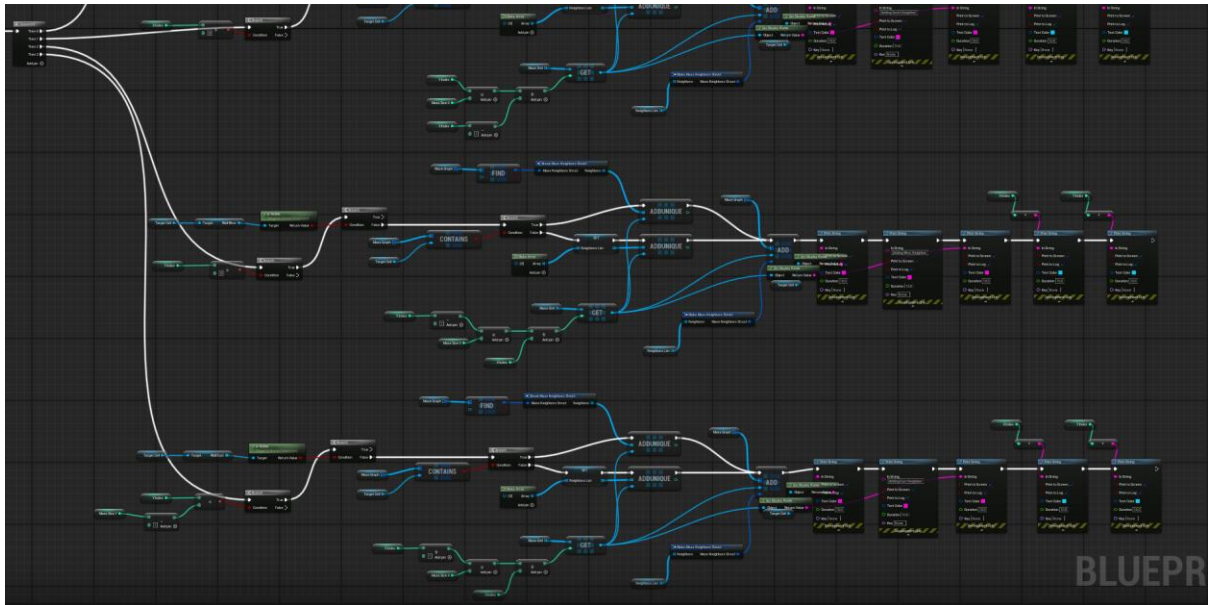


Ceci est considéré un algorithme parce qu'il traverse les données structurées (la grille du labyrinthe), effectue des vérifications de conditions spatiales, exécute un comportement (destruction ou placement des composants), préserve l'intégrité structurelle du labyrinthe et applique les contraintes. Il est efficace et s'exécute seulement une fois la génération du labyrinthe terminé.

Graph

Dans Whispers In The Dark, la connectivité entre les cellules n'est pas seulement un enjeu visuel, mais elle est aussi une structure logique indispensable à plusieurs systèmes de jeu, comme les décisions de l'intelligence artificielle lorsqu'il en est de chercher un chemin et de placer les murmures au bons endroits (à noter qu'en date de la remise de ce travail, cette partie n'a pas encore été faite, mais qu'elle est déjà planifiée d'utiliser ce graph). Pour gérer les relations entre les tuiles, une structure de données de graph appelée MazeGraph a été créée. Cette structure associe chaque cellule du labyrinthe à ses voisins valides, c'est-à-dire les cellules directement accessibles (non séparées par un mur). Le graph permet de modéliser l'évolution de la disposition du labyrinthe une fois que les murs sont ajoutés.





Contrairement à une liste ou une table de hachage, ce graph est idéal pour représenter les relations entre les cellules au lieu de des données statiques. Alors que MazeGrid stocke la position des tuiles, MazeGraph stocke la façon dont ces tuiles sont connectées.

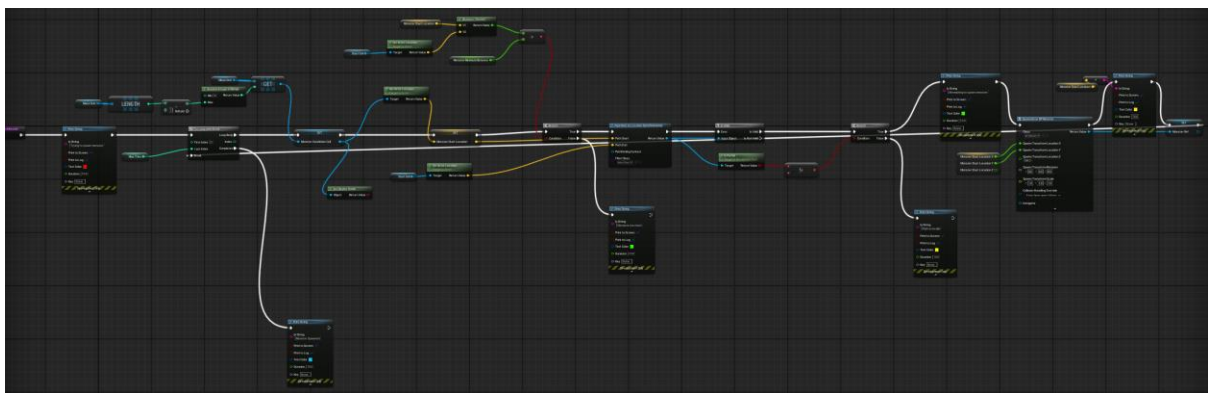
Essentiellement, les données dans MazeGraph sont stockés de cette manière :

$(3, 2) \rightarrow [(3,1), (3,2), (4,2)]$

Ceci signifie que (3,2) est navigable vers (3,1), (3,3) et (4,2), mais pas vers (2,2), qui est probablement bloqué par un mur. Le MazeGraph est construit immédiatement après la finalisation du placement des murs. Pour chaque cellule du MazeGrid, les coordonnées de ses voisins sont calculées. La table de hachage WallPresenceMap est vérifiée pour déterminer si un mur sépare la cellule de chaque tuile voisine. En l'absence de mur, le voisin est ajouté à la liste de voisins de la cellule. Initialement, MazeGrid et WallPresenceMap étaient seulement utilisés, mais ceux-ci ne permettent pas de vérifier toutes les connectivités entre les tuiles, ce qui nécessite une logique plus poussée et une liste dans une map. MazeGraph offre une méthode centralisée et réutilisable pour raisonner le mouvement, le chemin et la structure.

Troisième algorithme

Le monstre qui traque le joueur est un élément essentiel du jeu. Celui-ci par contre doit apparaître à un endroit valide, accessible et approprié au sein du labyrinthe généré procéduralement. Pour se faire, un algorithme d'apparition de monstre a été créé. Plutôt que de choisir un emplacement aléatoire, l'algorithme recherche intelligemment un point d'apparition qui répond à des critères spécifiques : il doit être situé à l'intérieur du labyrinthe, il doit pouvoir accéder au joueur, et il ne doit pas apparaître trop près de la position de départ du joueur.



L'algorithme fonctionne comme suit : on collecte toutes les cellules du labyrinthe en accédant à la grille du labyrinthe, on choisit ensuite une des cellules aléatoirement, on vérifie que la position n'est pas trop proche de la position de départ du joueur et si c'est correct, on effectue une vérification de chemin vers le joueur. Si le résultat est valide, on génère le monstre à l'emplacement aléatoire choisi. On se sert d'une boucle For avec un Break lorsque le monstre est généré.

Ceci est qualifié d'algorithme parce qu'on utilise un ensemble de cellules de spawn potentielles générées dynamiquement, qu'on traite ensuite à l'aide d'une boucle For et d'un branchement conditionnel à un emplacement valide. La boucle s'interrompt ensuite si une solution est trouvée. Il y a une prise de décision lorsqu'on intègre une condition claire de réussite ou d'échec. Il ne s'agit pas simplement d'un spawn, il s'agit d'un processus de filtrage et de sélection, mis en œuvre de manière structurée.

Conclusion

Quelles structures de données et quels algorithmes avez-vous appris ou approfondis ?

Ce projet m'a vraiment permis de comprendre le concept du graph d'une manière plus réelle que la théorie, ainsi que de comprendre son utilité et comment s'en servir.

Quels éléments du cours ont représenté les plus grands défis ?

Le plus difficile a été de développer un système procédural à la fois flexible et robuste. La génération procédurale est imprévisible, je devais donc m'assurer que les systèmes que je construisais étaient tolérants aux erreurs et s'auto-corrigeaient. Un autre défi majeur a été de gérer la complexité des Blueprints dans Unreal Engine. Les Blueprints étant visuels, des fonctions complexes comme la génération de labyrinthes ou la mise en place de murs peuvent devenir difficiles à déboguer ou à modifier. Il a fallu plusieurs itérations et un redémarrage complet du système de génération pour parvenir à une structure à la fois propre et fiable.

Quelle est la prochaine étape envisagée dans votre parcours académique ou professionnel ?

Trouver un stage sera la prochaine étape pour moi. Je dois déménager en même temps que le début du stage, donc ça me laisse plus de jeu quant à l'emplacement de celui-ci, puisque je pourrai déménager proche du stage et voir comment les équipes fonctionnent en vrai.

Quel projet réalisé par un ou une autre étudiante vous a particulièrement impressionné ?

Celui de Kseniya m'a vraiment impressionnée. Il y avait tellement de détails, un art et une histoire qui m'avait l'air réfléchi et fun à jouer.

Avez-vous l'intention de poursuivre le développement de votre projet final au-delà du cours ?

C'est certain qu'il me reste beaucoup de fonctionnalités à instaurer, et je tiens à venir à bout de ce jeu. Je pense avoir une base solide, mais les bugs multiples me donnent du fil à retordre question temps.

Voici ce qu'il reste à faire:

- Implémentation du path finding intelligent du monstre (la logique basée sur la lampe de poche est faite, mais mon monstre se déplace très mal)
- Instaurer les portes chuchotantes : sons spatiaux, audibles uniquement lorsque la lampe de poche est éteinte
- Ajout d'une condition de mort et d'un écran de mort lorsque le monstre attrape le joueur
- Ajout d'un système de bruits de pas
- Ajout d'un indicateur visuel ou d'un objet pour la sortie du labyrinthe (par exemple, un escalier qui mène à un écran de réussite)
- Menu principal (Démarrer la partie, Paramètres, Quitter)
- Écran de chargement pendant la génération du labyrinthe
- Court tutoriel
- Écran de fin/écran de victoire lorsque le joueur s'échappe
- Assurer que les portes qui spawn sont reliées à des murs pour pas qu'il y ait des portes en plein milieu de l'espace qui sont rattachées à rien.